WEEK 3 CAPSTONE PROJECT

PERSONAL EXPENSE TRACKER CLI

Python command-line application • CSV storage • Input validation • Spending summaries

1. Project Overview

This project is a terminal-based Personal Expense Tracker. It allows a user to record daily expenses, view saved transactions, calculate total spending by category, and keep records in a local CSV file. It uses Python's built-in csv, datetime, and os modules, so no external packages are required.

2. Features

Feature	What it does
Add expense	Records date, category, description, and amount.
View expenses	Displays all saved transactions in a readable table.
Spending summary	Calculates total expenses and totals for each category.
Input validation	Checks dates, empty fields, positive amounts, and menu choices.
Error handling	Uses try/except to handle invalid numbers and file errors.
CSV persistence	Loads existing records at startup and saves new records.

3. How to Run the Program

• Install Python 3.8 or newer if it is not already installed.

• Keep **expense_tracker.py** and **expenses.csv** in the same folder.

• Open a terminal in that folder.

• Run **python expense_tracker.py** (or **python3 expense_tracker.py** on some devices).

• Choose 1 to add an expense, 2 to view records, 3 to see the summary, or 4 to exit.

4. Sample CSV Data

The sample file uses four columns: date, category, description, and amount.

date	category	description	amount
2026-10-01	Food	Lunch	2500.00
2026-10-02	Transport	Bus fare	1200.00
2026-10-03	Food	Groceries	5800.00
2026-10-04	Education	Notebook	1800.00
2026-10-05	Transport	Taxi	3500.00

5. Expected Summary from Sample Data

Total expenses: 14,800.00
Food: 8,300.00
Transport: 4,700.00
Education: 1,800.00

6. Complete Python Source Code

Save the following code in a file named **expense_tracker.py**. The separate project ZIP contains the same runnable source file and sample CSV file.

```
"""
Personal Expense Tracker CLI
A simple command-line application for recording and reviewing expenses.

Run with: python expense_tracker.py
Data is saved automatically in expenses.csv in the same folder.
"""

import csv
import os
from datetime import datetime

DATA_FILE = "expenses.csv"
FIELDS = ["date", "category", "description", "amount"]


def load_expenses():
    """Load saved transactions. Return an empty list if no file exists."""
    expenses = []
    if not os.path.exists(DATA_FILE):
        return expenses

    try:
        with open(DATA_FILE, "r", newline="", encoding="utf-8") as file:
            reader = csv.DictReader(file)
            for row in reader:
                try:
                    row["amount"] = float(row["amount"])
                    expenses.append(row)
                except (ValueError, TypeError, KeyError):
                    # Ignore damaged rows rather than crashing the app.
                    continue
    except (OSError, csv.Error) as error:
        print(f"Could not read saved data: {error}")

    return expenses


def save_expenses(expenses):
    """Write all transactions to the CSV file."""
    try:
        with open(DATA_FILE, "w", newline="", encoding="utf-8") as file:
            writer = csv.DictWriter(file, fieldnames=FIELDS)
            writer.writeheader()
            for expense in expenses:
                writer.writerow({
                    "date": expense["date"],
                    "category": expense["category"],
                    "description": expense["description"],
                    "amount": f'{expense["amount"]:.2f}'
                })
    except OSError as error:
        print(f"Could not save data: {error}")


def add_expense(expenses):
    """Collect and validate a new expense."""
    print("\n--- Add an expense ---")

    date_text = input("Date (YYYY-MM-DD, leave blank for today): ").strip()
    if not date_text:
        date_text = datetime.now().strftime("%Y-%m-%d")

    try:
        datetime.strptime(date_text, "%Y-%m-%d")

    except ValueError:
        print("Invalid date. Use the format YYYY-MM-DD.")
        return

    category = input("Category (e.g. Food, Transport): ").strip().title()
    if not category:
        print("Category cannot be empty.")
        return

    description = input("Description: ").strip()
    if not description:
        print("Description cannot be empty.")
        return

    try:
        amount = float(input("Amount: ").strip().replace(",", ""))
        if amount <= 0:
            print("Amount must be greater than zero.")
```

```
            return
        except ValueError:
            print("Invalid amount. Enter a number, such as 2500 or 2500.50.")
            return

        expenses.append({
            "date": date_text,
            "category": category,
            "description": description,
            "amount": amount
        })
        save_expenses(expenses)
        print("Expense saved successfully!")


    def view_expenses(expenses):
        """Display all saved expenses."""
        print("\n--- All expenses ---")
        if not expenses:
            print("No expenses have been recorded yet.")
            return

        print(f'{"DATE":<12} {"CATEGORY":<16} {"DESCRIPTION":<25} {"AMOUNT":>12}')
        print("-" * 69)
        for item in expenses:
            print(
                f'{item["date"]:<12} {item["category"]:<16} '
                f'{item["description"][:24]:<25} {item["amount"]:>12,.2f}'
            )


    def show_summary(expenses):
        """Calculate total spending and totals grouped by category."""
        print("\n--- Spending summary ---")
        if not expenses:
            print("No data available for a summary.")
            return

        totals = {}
        grand_total = 0.0

        for item in expenses:
            category = item["category"]
            totals[category] = totals.get(category, 0.0) + item["amount"]
            grand_total += item["amount"]

        print(f"Total expenses: {grand_total:,.2f}")

        print("\nSpending by category:")
        for category, amount in sorted(totals.items()):
            print(f"  {category:<20} {amount:>12,.2f}")


    def main():
        """Run the application's main menu."""
        expenses = load_expenses()
        print("================================")
        print("     PERSONAL EXPENSE TRACKER")
        print("================================")

        while True:
            print("\n1. Add expense")
            print("2. View all expenses")
            print("3. View spending summary")
            print("4. Exit")

            choice = input("Choose an option (1-4): ").strip()

            if choice == "1":
                add_expense(expenses)
            elif choice == "2":
                view_expenses(expenses)
            elif choice == "3":
                show_summary(expenses)
            elif choice == "4":
                print("Thank you for using Personal Expense Tracker.")
                break
            else:
                print("Invalid choice. Please enter 1, 2, 3, or 4.")


    if __name__ == "__main__":
        main()
```

7. Three-Minute Live Walkthrough Script

0:00–0:30 — Introduction

Introduce the project as a Python CLI tool that records expenses and stores them in a CSV file.

0:30–1:10 — Add a transaction

Choose option 1. Enter a date (or leave it blank for today), category, description, and a positive amount. Explain that the program validates the input and saves it.

1:10–1:40 — View transactions

Choose option 2 and point out the saved date, category, description, and amount.

1:40–2:20 — Show the summary

Choose option 3. Explain that the program adds amounts together and groups spending by category.

2:20–2:45 — Demonstrate error handling

Try an invalid menu option or a non-numeric amount. Explain that try/except prevents the program from crashing.

2:45–3:00 — Persistence and close

Exit and run the program again. Show that earlier records remain because they were stored in expenses.csv.

8. Project Checklist

✓ Daily transactions can be recorded.

✓ Spending totals are calculated by category.

✓ Invalid inputs are handled with try/except blocks and clear messages.

✓ Transactions are saved to and loaded from a local CSV file.

✓ A sample CSV file is included.

✓ A three-minute presentation walkthrough is prepared.